Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 09 — Estado da Aplicação e o Hook useState

Por que a tela não atualiza sozinha — e como avisar o React que algo mudou

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

O Problema: A Tela Não Atualiza

```
let contador = 0;

function incrementar() {
  contador = contador + 1;
  console.log(contador);
}

<Text>{contador}</Text>
```

O que acontece de verdade

O console mostra 1, 2, 3, 4... a variável muda de verdade. Mas o número na tela continua travado em 0, pra sempre.

O React só redesenha a tela quando alguém avisa explicitamente que algo mudou. Mudar uma variável comum é invisível pra ele.

CONCEITO

O Que É "Estado" (State)?

Estado é qualquer dado que muda com o tempo e que a interface precisa refletir na tela. É a "memória" de um componente entre uma interação e outra.

- O texto que o usuário já digitou
- Se uma opção está marcada ou não
- Quantos itens estão no carrinho
- Qual aba está selecionada agora

Nos exemplos que já fizemos (Aulas 05 a 08), tudo era fixo: arrays, textos e estilos definidos de antemão. A partir de hoje, a interface passa a reagir ao que acontece durante o uso do app.

CONCEITO

Variável Comum x Estado do React

VARIÁVEL COMUM (let / const)

- Mudar o valor não avisa o React
- A tela não é redesenhada
- Em alguns casos, o valor volta ao original no próximo render
- Serve só pra cálculos que não aparecem na tela

ESTADO (useState)

- Mudar o valor (do jeito certo) avisa o React
- A tela é redesenhada automaticamente
- O valor sobrevive entre um render e outro
- Serve pra qualquer dado que aparece ou influencia a tela

Apresentando o Hook useState

```
import { useState } from 'react';

const [contador, setContador]
  = useState(0);
```

contador — o valor atual

É só pra leitura. Nunca se altera esse nome diretamente.

setContador — a função que atualiza

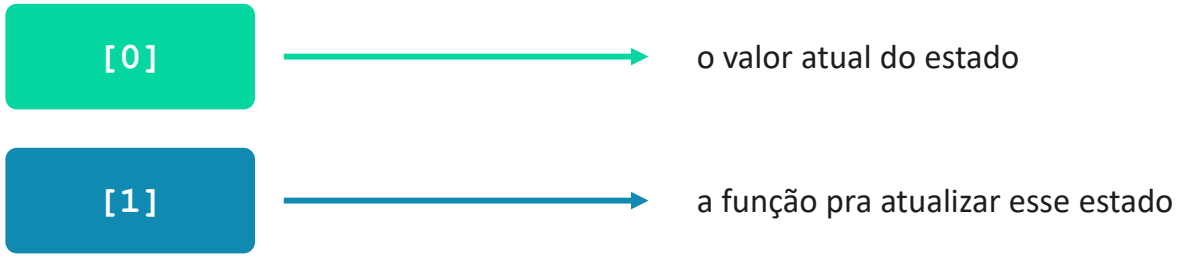
Chamar essa função é o único jeito certo de mudar o valor.

useState(0) recebe o valor inicial do estado — nesse caso, o contador começa em 0. Esse valor inicial só é usado uma vez, na primeira vez que o componente é montado.

O Retorno do useState É Um Array

Lá na Aula 08 desestruturamos objetos ({ nome, curso } = aluno). Agora é a mesma ideia, mas com um array — e em array o que importa é a posição, não o nome da propriedade.

```
useState(0)  →  retorna [0, funçãoSet]
```



const [contador, setContador] = useState(0) — os nomes contador e setContador são escolha nossa, a posição no array é o que define o papel de cada um.

O Ciclo do Estado

1

Usuário interage

toca no botão, digita no campo...

2

Chama setContador(novoValor)

avisa o React que o estado mudou

3

React re-renderiza o componente

roda a função do componente de novo

4

Tela mostra o novo valor

a interface finalmente reflete a mudança

Antes x Depois: o Mesmo Contador

VARIÁVEL COMUM — não funciona

```
let contador = 0;

function incrementar() {
  contador = contador + 1;
}

<Text>{contador}</Text>
```

useState — funciona

```
const [contador, setContador]
  = useState(0);

function incrementar() {
  setContador(contador + 1);
}

<Text>{contador}</Text>
```

Regra de Ouro: Nunca Altere o Estado Direto

ERRADO

```
contador = contador + 1;
contador++;
```

Muda a variável, mas o React não fica sabendo. A tela não atualiza.

CERTO

```
setContador(contador + 1);
```

Chama a função set. O React recebe o aviso e redesenha a tela com o novo valor.

Trate a variável do useState quase como "somente leitura": ela só existe pra ser exibida. Quem tem permissão de mudar o estado é sempre a função set correspondente.

De Estático para Controlado: o TextInput

AULA 08 — sem estado

```
<TextInput
  placeholder="Seu nome"
  onChangeText={(t) =>
    console.log(t)
  }
/>
```

HOJE — controlado por estado

```
const [nome, setNome] =
  useState('');

<TextInput
  value={nome}
  onChangeText={setNome}
/>
```

Sem value, o TextInput guarda o texto por conta própria — não temos acesso a esse texto fora dele. Com value + onChangeText apontando pro estado, é a nossa tela que passa a controlar (e conhecer) o que está escrito.

CONCEITO

Quantos Estados Eu Posso Ter?

Quantos precisar. Cada chamada de useState cria um estado independente — eles não se misturam.

```
const [nome, setNome] = useState('');
const [cidade, setCidade] = useState('');
const [aceito, setAceito] = useState(false);
```

Convenção de nomes: valor / setValor. Não é obrigatório, mas deixa qualquer pessoa lendo o código sabendo, só pelo nome, qual função atualiza qual estado.

O Que Levamos de Hoje

- Estado é o dado que muda com o tempo e que a interface precisa refletir
- Uma variável comum não avisa o React — mudar ela não redesenha a tela
- useState(inicial) devolve um array: [valor, funçãoSet]
- Chamar setValor(novo) é o que dispara o re-render do componente
- Nunca altere o valor do estado direto — sempre use a função set

Próxima aula: seguimos avançando no cronograma — conteúdo definido de acordo com o ritmo da turma.